



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 7/4 ч.

Разработка мобильных приложений

(наименование дисциплины (модуля) в соответствии с учебным планом)

Уровень

бакалавриат

(бакалавриат, магистратура, специалитет)

Форма обучения

очная

(очная, очно-заочная, заочная)

Направление(-я)

подготовки

09.03.02 «Информационные системы и технологии»

(код(-ы) и наименование(-я))

Институт

кибербезопасности и цифровых технологий

(полное и краткое наименование)

Кафедра

КБ-14 «Цифровые технологии обработки данных»

(полное и краткое наименование кафедры, реализующей дисциплину (модуль))

Используются в данной редакции с учебного года

2024/25

(учебный год цифрами)

Проверено и согласовано «___» _____ 20__ г.

(подпись директора Института/Филиала
с расшифровкой)

Москва 2024 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1. VIEW BINDING.....	4
2. ПРИНЦИПЫ НАВИГАЦИИ.....	5
3. ВИДЫ НАВИГАЦИИ.....	9
4. NAVIGATION COMPONENT.....	10
5. BOTTOM NAVIGATION.....	15
6. NAVIGATION DRAWER.....	19
7. КОНТРОЛЬНОЕ ЗАДАНИЕ.....	24

ВВЕДЕНИЕ

Навигация является одним из ключевых элементов в разработке Android-приложений, обеспечивая пользователю интуитивный и удобный способ перемещения между экранами приложения. В современных приложениях с все более насыщенными функциональными возможностями правильно организованная навигация заметно влияет на качество пользовательского опыта. Навигация в Android включает в себя множество подходов, от простого переключения между экранами с помощью Activity до более сложных сценариев, таких как использование фрагментов, бэк-стека и глубоких ссылок (deep links). Основная задача навигации - организовать переходы между экранами таким образом, чтобы пользователи могли легко находить и использовать нужные функции приложения. При этом важно учитывать не только визуальные аспекты, но и технические, такие как сохранение состояния и адаптация к различным устройствам. Существует несколько основных видов навигации, которые применяются в зависимости от структуры приложения.

Линейная навигация предполагает последовательные переходы между экранами, где каждый новый экран заменяет предыдущий в стеке. Реализуется это с помощью Activity и методов `startActivity()` или Fragment в связке с FragmentManager.

Иерархическая навигация организована в виде дерева, где есть корневой и вложенные экраны. Пользователь может углубляться в детали и возвращаться к корневому экрану. Примером может служить приложение «Настройки», корневой экран содержит список категорий, а каждая категория открывает свои настройки.

Навигация с использованием шторки (Navigation Drawer) представляет собой боковое меню, открывающееся свайпом или нажатием на значок «гамбургера». Она предназначена для переходов между разделами приложения, как, например, в приложении Gmail, где можно выбрать «Входящие», «Отправленные» и др.

Навигация через нижнее меню (Bottom Navigation) удобна для приложений с 3-5 основными экранами. Нижняя панель содержит вкладки для быстрого переключения между разделами.

Табовая навигация (Tab Navigation) предполагает верхнюю панель с вкладками для переключения между категориями.

Модальная навигация предусматривает открытие диалоговых окон или временных экранов поверх текущего. Она может использоваться для подтверждения удаления или входа в систему.

С развитием экосистемы Android появился *Navigation Component* - мощный инструмент, который упрощает управление навигацией и обеспечивает более структурированный подход к ее реализации. Он поддерживает современные паттерны проектирования интерфейсов, что делает его универсальным решением для разработки.

1. VIEW BINDING

View Binding — это механизм, предоставляемый Android для упрощённого и безопасного взаимодействия с элементами пользовательского интерфейса. Он заменяет использование метода *findViewById()* и минимизирует вероятность ошибок, связанных с несоответствием идентификаторов элементов интерфейса. Он автоматически генерирует binding-классы (классы привязки) для XML-разметки в проекте, которые позволяют напрямую обращаться к элементам разметки через свойства. Чтобы использовать View Binding, необходимо включить его в *build.gradle*:

```
android {  
    viewBinding {  
        enable = true  
    }  
}
```

После этого для каждой XML-разметки автоматически создается binding-класс, каждый файл разметки соответствует отдельному классу привязки. Имя класса формируется на основе имени файла разметки и преобразуется в CamelCase, например *activity_main.xml* преобразуется в *ActivityMainBinding*. Стоит учитывать, что View Binding не работает с элементами, созданными динамически. Класс привязки создаётся с помощью метода *inflate()*, элементы интерфейса становятся доступными как свойства объекта привязки.

```
public class MainActivity extends AppCompatActivity {  
    private ActivityMainBinding binding;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        // ...  
        binding = ActivityMainBinding.inflate(getLayoutInflater());  
        setContentView(binding.getRoot());  
        binding.textView.setText("View Binding");  
    }  
}
```

2. ПРИНЦИПЫ НАВИГАЦИИ

Навигация между различными экранами и приложениями является основной частью пользовательского опыта. Следующие принципы задают основу для последовательного и интуитивно понятного пользовательского опыта во всех Android-приложениях.

Фиксированный пункт назначения. Каждое приложение имеет фиксированный начальный пункт назначения - это первый экран, который пользователь видит при запуске приложения из меню списка приложений. Он также является последним экраном, который пользователь видит, когда возвращается в меню списка приложений после нажатия кнопки «Back». Рассмотрим пример.

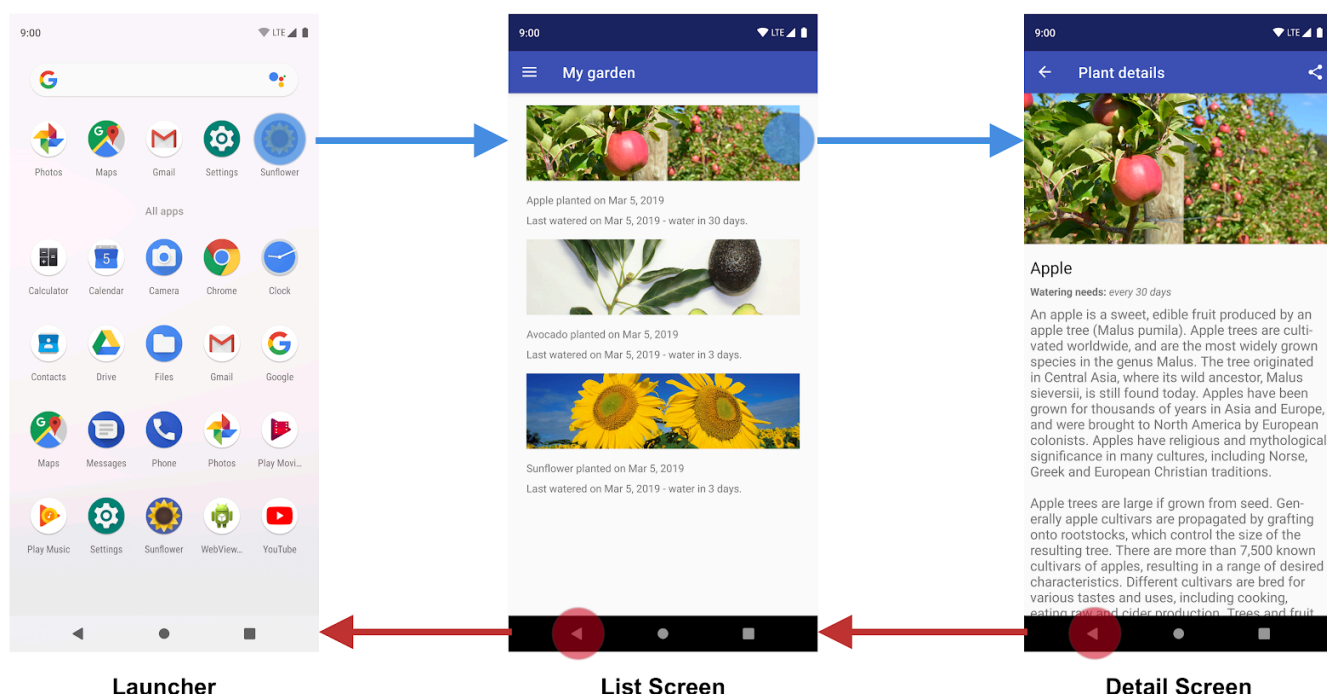


Рисунок 1. Порядок отображения экранов приложения

При запуске приложения из меню списка приложений первым экраном, который видит пользователь, является экран со списком растений. Он же является и последним экраном, который пользователь видит перед выходом из приложения. Если нажать кнопку «Back» на экране списка, пользователь вернется в меню списка приложений.

Состояние навигации представлено в виде стека пунктов назначения. Когда приложение запускается впервые, для пользователя создается новая задача (набор действий, с которыми взаимодействуют пользователи, пытаясь сделать что-то в приложении), и приложение отображает ее начальный пункт назначения. Он

становится базовым пунктом назначения в бэк-стеке и является основой для состояния навигации приложения. Верхняя часть стека - это текущий экран, а предыдущие пункты назначения в стеке представляют собой историю того, где пользователь уже побывал, в нижней части стека всегда находится начальный пункт назначения приложения. Операции, изменяющие бэк-стек, всегда выполняются в верхней части стека, либо добавляя новый пункт назначения на вершину стека, либо удаляя самый верхний пункт назначения из стека. Переход к пункту назначения перемещает его на вершину стека.

Идентичное поведение кнопок Up и Back. Кнопка «Back» отображается на панели навигации системы в нижней части экрана и используется для навигации в обратном хронологическом порядке по истории экранов, с которыми пользователь недавно работал. При нажатии кнопки «Back» текущий пункт назначения переносится в верхнюю часть бэк-стека, и происходит переход к предыдущему пункту назначения. Кнопка «Up» отображается на панели приложений в верхней части экрана. В рамках задачи приложения кнопки «Up» и «Back» ведут себя одинаково.



Рисунок 2. Кнопки «Up» и «Back»

Кнопка «Up» никогда не закрывает приложение. Если пользователь находится в начальной точке приложения, то кнопка «Up» не появляется, а кнопка «Back» отображается и приводит к выходу из приложения. Когда приложение запускается с помощью глубокой ссылки, кнопка «Up» переводит пользователя обратно на задачу приложения через имитацию бэк-стека, а не на приложение, вызвавшее глубокую ссылку, в отличие от кнопки «Back».

Глубокие ссылки имитируют ручную навигацию. При глубокой ссылке (механизм, позволяющий открывать определенные экраны приложения через URL-ссылки) или ручном переходе к определенному пункту назначения можно использовать кнопку «Up» для навигации по пунктам назначения обратно к начальному пункту назначения. При глубоком переходе (deep linking) к пункту назначения все существующие бэк-стеки удаляются и заменяются бэк-стеком глубокой ссылки. На примере рассматриваемого приложения предположим, что пользователь ранее запустил приложение с меню списка приложений и перешел на экран детальной информации. Если посмотреть на бэк-стек, то можно увидеть, что самым верхним экраном является экран детальной информации.

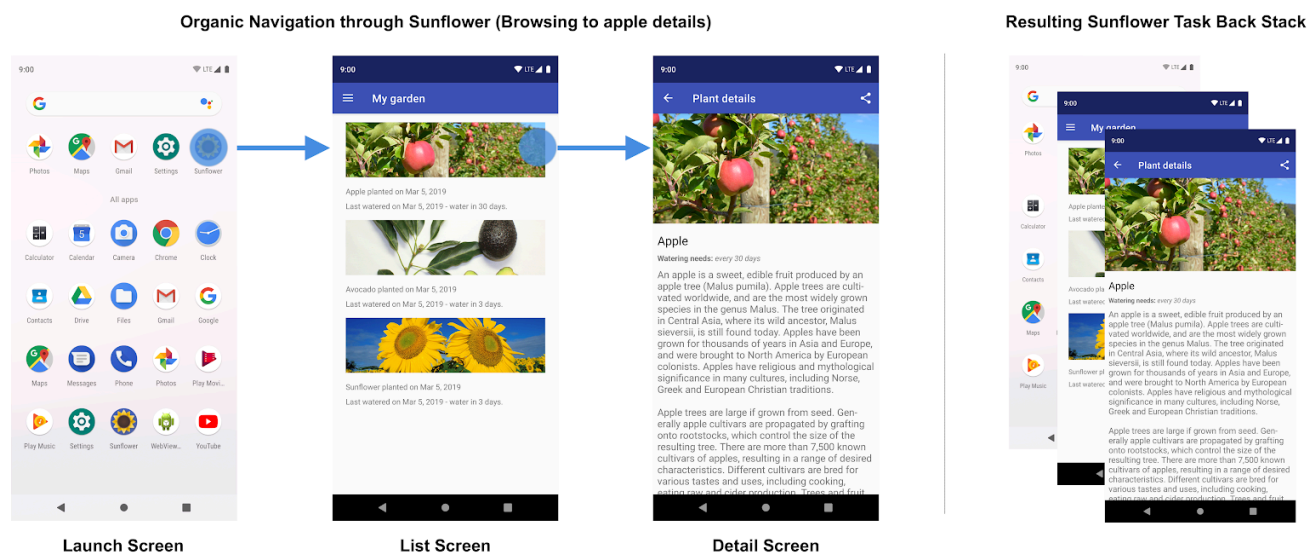


Рисунок 3. Навигация пользователя по приложению и состояние бэк-стека

В этот момент пользователь может нажать кнопку Home, чтобы перевести приложение в фоновый режим. Далее, предположим, в приложении есть поддержка глубокой ссылки, которая позволяет пользователям переходить непосредственно к экрану с подробной информацией о конкретном элементе. Открытие приложения по этой глубокой ссылке полностью заменяет текущий бэк-стек, показанный на рисунке 3, новым бэк-стеком, как показано на рисунке 4.

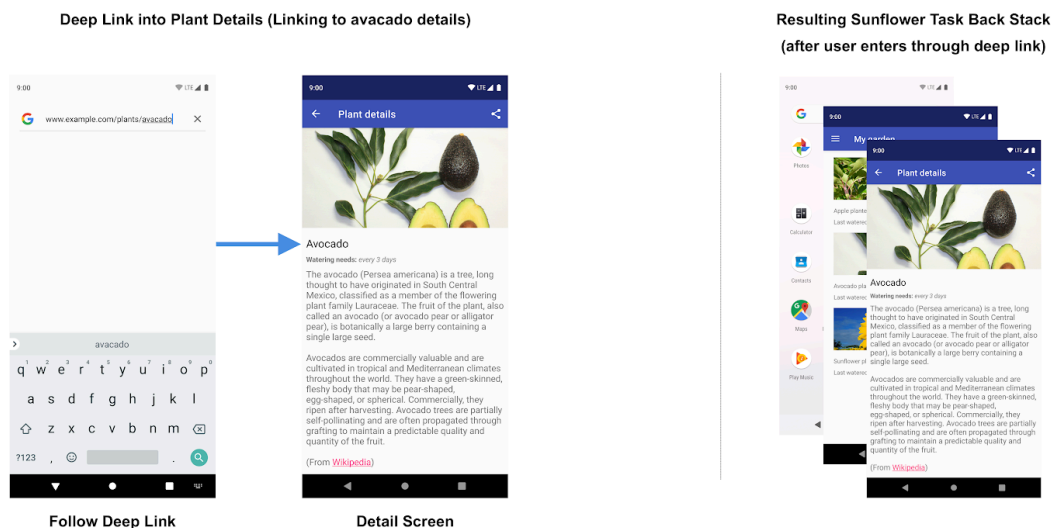


Рисунок 4. Переход по глубокой ссылке заменяет существующий бэк-стек

Заметим, что на месте бэк-стека появился синтетический бэк-стек с экраном детальной информации об авокадо. Экран списка, который является начальным

пунктом назначения, также был добавлен в бэк-стек. Оригинальный бэк-стек исчез, включая информацию приложения о том, что пользователь ранее находился на экране детальной информации о яблоке. Это связано с тем, что синтетический бэк-стек должен быть реалистичным, он должен соответствовать бэк-стеку, который мог бы быть достигнут при обычной навигации по приложению.

3. ВИДЫ НАВИГАЦИИ

Навигация с использованием Activity представляет собой переходы между экранами, где каждый экран реализован в виде отдельной Activity, часто используется в простых приложениях. Для навигации между Activity используется Intent, он может быть явным, указывая целевую Activity, или неявным, описывая действие, которое должно быть выполнено, предоставляя системе возможность выбрать подходящую Activity. Для передачи данных между Activity используются Intent Extras, данные передаются в формате ключ-значение, где ключ представляет собой строку, а значение может быть различного типа. Если одна Activity запускает другую и ожидает результата, используется метод *startActivityForResult* или Activity Result API. Данный способ прост в реализации, так как каждая Activity независима и имеет собственный жизненный цикл, обеспечивается разделение задач между экранами. Однако в таком случае сложнее создавать адаптивные макеты и гибко управлять бэк-стеком.

Навигация с использованием фрагментов предоставляет более гибкий способ организации пользовательского интерфейса и навигации внутри одной Activity. Фрагмент представляет собой компонент, который можно добавлять, заменять или удалять внутри Activity, обеспечивая модульное управление интерфейсом и его обновлением. Это позволяет реализовывать более сложные интерфейсы. Каждый фрагмент имеет собственный жизненный цикл, который тесно связан с жизненным циклом его Activity. Для управления фрагментами используется FragmentManager, предоставляющий методы для выполнения операций с фрагментами. Эти операции фиксируются в виде транзакций с помощью FragmentTransaction. Например, текущий фрагмент можно заменить новым, а предыдущий сохранить в бэк-стеке, чтобы пользователь мог вернуться к нему позже. Передача данных между фрагментами может осуществляться с помощью ViewModel или Fragment Result API.

Deep Links (глубокие ссылки). Это механизм, позволяющий открывать определенные экраны приложения из внешних источников, таких как веб-сайты, электронная почта и др. Он реализует плавный пользовательский опыт, направляя пользователей сразу к необходимому контенту, обеспечивают связь между веб- и мобильной версиями сервиса, удобен для привлечения пользователей.

4. NAVIGATION COMPONENT

Android Navigation Component — это библиотека от Google, предназначенная для упрощения реализации навигации в приложениях. Она позволяет реализовывать рассмотренные принципы навигации по умолчанию, гарантируя, что пользователи смогут применять одни и те же шаблоны в навигации при переходе между приложениями. К преимуществам относятся:

- использование стандартизированных ресурсов для анимаций и переходов;
- обработка глубоких ссылок, которые перенаправляют пользователя непосредственно к пункту назначения;
- поддержка шаблонов пользовательского интерфейса (Navigation Drawer и Bottom navigation) с минимальными затратами;
- обеспечение безопасной передачи данных между пунктами назначения;
- использование ViewModel с графом навигации для совместного доступа к данным;
- встроенная поддержка транзакций фрагментов;
- корректная обработка действий «Back» и «Up» по умолчанию.

В таблице приведены ключевые термины, используемые в Navigation Component.

Термин	Описание	Тип
Host (хост)	Элемент пользовательского интерфейса (контейнер), в котором отображаются пользовательские экраны	NavHostFragment
Graph (граф)	Структура данных, определяющая все пункты назначения в приложении и то, как они связаны между собой	NavGraph
Controller (контроллер)	Центральный координатор для управления навигацией между пунктами назначения, предоставляет методы для навигации	NavController
Destination (пункт назначения)	Узел в графе навигации, при переходе пользователя к этому узлу хост отображает его содержимое	NavDestination
Route (маршрут)	Абстрактное обозначение пути, по которому можно перейти к конкретному пункту назначения	Любой сериализуемый тип данных

Чтобы использовать Navigation Component, необходимо подключить его в *build.gradle*.

```
val nav_version = "2.8.4"
```

```
implementation("androidx.navigation:navigation-fragment:$nav_version")
implementation("androidx.navigation:navigation-ui:$nav_version")
```

Навигационный контроллер - одна из ключевых концепций навигации. Он хранит навигационный граф и предоставляет методы, позволяющие вашему приложению перемещаться между пунктами назначения в графе. При использовании Navigation component возможно создать контроллер навигации с помощью класса NavController. Он отслеживает, какие пункты назначения посетил пользователь, и позволяет ему перемещаться между ними. Чтобы получить объект NavController необходимо сначала получить NavHostFragment, а затем получить из него NavController.

```
NavHostFragment navHostFragment = (NavHostFragment)
getSupportFragmentManager()
    .findFragmentById(R.id.nav_host_fragment);
NavController navController = navHostFragment.getNavController();
```

Navigation component использует навигационный граф для управления навигацией приложения. Граф навигации - это структура данных, содержащая каждый пункт назначения в приложении и связи между ними. Стоит отметить, что граф навигации отличается от бэк-стека. В таблице представлены три основных типа пунктов назначения, используемых при навигации.

Тип	Описание	Пример использования
Hosted	Заполняет весь навигационный хост, то есть размер размещенного пункта назначения совпадает с размером навигационного хоста, а предыдущие пункты назначения не видны.	Основной и подробный экраны.
Dialog	Представляет наложенные компоненты пользовательского интерфейса. Интерфейс не привязан к местоположению навигационного хоста или его размеру. Предыдущие пункты назначения видны под пунктом назначения.	Оповещения, формы.
Activity	Представляет уникальные экраны или функции в приложении.	Служат точкой выхода в навигационный граф, который запускает новую Activity, которая управляется отдельно от Navigation component.

Существует несколько способов создания навигационного графа с использованием фрагментов:

- с помощью Kotlin DSL программно создать NavGraph и напрямую применить его к фрагменту NavHostFragment;
- описать навигационный хост и граф в XML;
- использовать редактор GUI в Android Studio для создания и настройки графа в виде файла ресурсов XML.

Стоит отметить, что Navigation component предназначен для приложений, в которых есть одна основная Activity с несколькими пунктами назначения (фрагментами). Она связана с навигационным графом и содержит фрагмент NavHostFragment, который отвечает за смену пункта назначения. Рассмотрим создание графа путем его описания в XML. Сначала создадим фрагмент NavHostFragment, он служит в качестве хоста навигации, который содержит навигационный граф.

```
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent">
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:navGraph="@navigation/nav_graph" />
</FrameLayout>
```

Фрагмент NavHostFragment содержит атрибут *app:navGraph*, который необходим для подключения навигационного графа к хосту навигации. Опишем сам граф в директории *navigation*.

```
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:id="@+id/nav_graph"
app:startDestination="@id/fragmentOne">

<fragment
    android:id="@+id/fragmentOne"
    android:name="com.mirea.navigation.FragmentOne"
    android:label="Fragment one">
```

```

        <action
            android:id="@+id/action_fragmentOne_to_fragmentTwo"
            app:destination="@id/fragmentTwo" />
    </fragment>

    <fragment
        android:id="@+id/fragmentTwo"
        android:name="com.mirea.navigation.FragmentTwo"
        android:label="Fragment two" />
</navigation>

```

Действия используются для определения связей между различными пунктами назначения. В данном случае `FragmentOne` содержит действие, которое осуществляет переход к `FragmentTwo`. Для перехода к пункту назначения следует использовать метод `NavController.navigate()`, у которого существует множество перегрузок. Выбор перегрузки зависит от конкретного контекста. Чтобы перейти к пункту назначения по идентификатору, требуется использовать `navigate(int)`. Он принимает идентификатор либо ресурса, либо действия, либо пункта назначения.

```
Navigation.findNavController(view).navigate(R.id.navigationId);
```

При навигации с помощью идентификаторов следует по возможности использовать действия. Они предоставляют дополнительную информацию в навигационном графе, наглядно показывая, как ваши пункты назначения связаны друг с другом.

`NavController` хранит бэк-стек, содержащий пункты назначения, которые посетил пользователь. По мере того как пользователь перемещается по экранам в приложении, `NavController` добавляет и удаляет пункты назначения из бэк-стека. Бэк-стек представляет собой структуру данных по принципу «last in, first out», поэтому `NavController` добавляет элементы в верхнюю часть стека и удаляет их из нее. Когда пользователь открывает приложение, `NavController` перемещает первый пункт назначения на вершину бэк-стека. Каждый вызов `NavController.navigate()` добавляет заданный пункт назначения на вершину стека. При нажатии кнопки «Up» или «Back» вызываются методы `NavController.navigateUp()` и `NavController.popBackStack()` соответственно. Для перехода к определенному пункту назначения возможно использовать метод `popBackStack()`. Важно отметить, что он выгружает все, что находится в стеке над этим пунктом назначения. Метод также принимает аргумент `inclusive`, который определяет, должен ли `NavController` также удалить указанный пункт назначения из бэк-стека после перехода к нему.

```
navController.popBackStack(R.id.destinationId, true);
```

В данном примере `NavController` переходит к пункту назначения с идентификатором *destinationId*. Поскольку значение аргумента *inclusive* истинно, `NavController` также убирает данный пункт назначения из бэк-стека. Когда функция `popBackStack()` возвращает `false`, последующий вызов `NavController.getCurrentDestination()` возвращает `null`. Это означает, что приложение удалило последний пункт назначения из стека. В этом случае пользователь видит только пустой экран. Это может произойти либо если `popBackStack()` ничего не удалил из стека, либо если он удалил пункт назначения, и теперь стек пуст. Чтобы решить эту проблему, необходимо перейти к новому пункту назначения или вызвать функцию `finish()` для завершения `Activity`.

```
if (!navController.popBackStack()) {  
    finish();  
}
```

5. BOTTOM NAVIGATION

Bottom Navigation - это один из популярных типов навигации, который предоставляет пользователю доступ к основным разделам через панель, расположенную в нижней части экрана. Подход особенно удобен для приложений, где пользователь часто переключается между несколькими ключевыми экранами, он обеспечивает быстрый доступ, улучшает пользовательский опыт и соответствует современным стандартам дизайна.

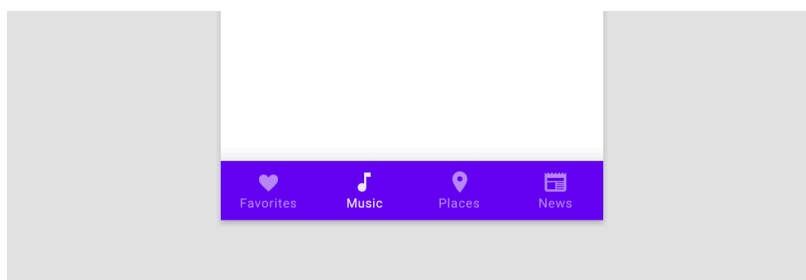


Рисунок 6. Пример интерфейса с Bottom Navigation

Центральным элементом является нижняя панель навигации, которая содержит пункты меню в виде иконок и текстовых подписей. Каждый пункт обычно связан с определённым фрагментом или экраном приложения, который отображает содержимое выбранного раздела. Элементы меню определяются в отдельном файле, где указываются подписи, иконки и идентификаторы. Использование Bottom Navigation делает приложение удобным для использования одной рукой, особенно на устройствах с большими экранами. Это решение соответствует рекомендациям Material Design, которые рекомендуют использование панели для переключения между 3–5 ключевыми разделами приложения. При наличии более трёх пунктов текстовые подписи обычно скрываются для неактивных элементов, оставляя только иконки. Важной особенностью является поддержка визуальных эффектов (изменение цвета, размера иконки, текста активного элемента), а также наличие анимаций. Bottom Navigation может быть использован в сочетании с глубокой навигацией, что позволяет сохранять состояние активного раздела при переходе на другие экраны. Рассмотрим основные элементы Bottom Navigation.

BottomNavigationView - основной компонент для реализации нижней панели навигации, который отображает интерактивные элементы, представляющие разделы приложения.

Элементы меню включает в себя иконку, текст (опционально), уникальный идентификатор для обработки действий. *BottomNavigationView* автоматически выделяет активный пункт меню, чтобы пользователь всегда понимал состояние навигации.

Основной контент - центральная область экрана, где отображается содержимое приложения в зависимости от выбранного пункта меню.

Для настройки Bottom Navigation обычно создается разметка с элементом панели и контейнером для фрагментов, а пункты меню описываются в отдельном файле. В коде приложения обрабатываются действия пользователя при выборе элементов меню, которые обновляют отображаемое содержимое. Подготовим 3 фрагмента: HomeFragment, InfoFragment, ProfileFragment. Для перехода между ними будет реализована навигация. Пример класса фрагмента приведен ниже.

```
public class HomeFragment extends Fragment {
    private FragmentHomeBinding binding;
    public View onCreateView(@NonNull LayoutInflater inflater,
                             ViewGroup container,
                             Bundle savedInstanceState) {
        binding = FragmentHomeBinding
            .inflate(inflater, container, false);
        return binding.getRoot();
    }
    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }
}
```

Опишем элементы меню в *menu/menu.xml*.

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/navigation_home"
        android:icon="@drawable/ic_launcher_foreground"
        android:title="Home" />
    <item
        android:id="@+id/navigation_info"
        android:icon="@drawable/ic_launcher_foreground"
        android:title="Info" />
    <item
        android:id="@+id/navigation_profile"
        android:icon="@drawable/ic_launcher_foreground"
        android:title="Profile" />
</menu>
```

Опишем навигацию между фрагментами в *navigation/navigation.xml*.

```

<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/navigation"
    app:startDestination="@+id/navigation_home">
    <fragment
        android:id="@+id/navigation_home"
        android:name="com.mirea.bottomnavigation.HomeFragment"
        android:label="Home"
        tools:layout="@layout/fragment_home" />
    <fragment
        android:id="@+id/navigation_info"
        android:name="com.mirea.bottomnavigation.InfoFragment"
        android:label="Info"
        tools:layout="@layout/fragment_info" />
    <fragment
        android:id="@+id/navigation_profile"
        android:name="com.mirea.bottomnavigation.ProfileFragment"
        android:label="Profile"
        tools:layout="@layout/fragment_profile" />
</navigation>

```

В разметке Activity используем компоненты *BottomNavigationView* и *fragment*.

```

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingTop="?attr/actionBarSize">
    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/nav_view"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:background="?android:attr/windowBackground"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:menu="@menu/menu" />
    <fragment
        android:id="@+id/nav_host"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="match_parent"

```

```

        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toTopOf="@id/nav_view"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/navigation" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

В классе Activity опишем навигацию с помощью Navigation Component.

```

public class MainActivity extends AppCompatActivity {
    private ActivityMainBinding binding;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        AppBarConfiguration appBarConfiguration =
            new AppBarConfiguration.Builder(
                R.id.navigation_home,
                R.id.navigation_info,
                R.id.navigation_profile)
                .build();
        NavController navController = Navigation.findNavController(
            this, R.id.nav_host);
        NavigationUI.setupActionBarWithNavController(this,
            navController, appBarConfiguration);
        NavigationUI.setupWithNavController(binding.navView,
            navController);
    }
}

```

Для создания проекта с использованием Bottom Navigation средствами IDE можно воспользоваться «*New - New module - Bottom Navigation Views Activity*».

Задание. Продумайте концепцию приложения, реализуйте его навигацию с помощью Bottom Navigation, добавьте иконки, обновите цветовую гамму. Название модуля «BottomNavigationApp».

6. NAVIGATION DRAWER

Navigation Drawer (боковая шторка) - это компонент пользовательского интерфейса, предоставляющий боковое меню для навигации между основными разделами приложения. Он часто используется для реализации доступа к многочисленным разделам, обеспечивая удобство и интуитивность интерфейса. Основные преимущества Navigation Drawer включают удобство в использовании, экономию пространства, гибкость в настройке меню и минимизацию отвлекающих элементов на экране, так как меню остаётся скрытым, пока пользователь не откроет его. Этот подход позволяет легко организовать навигацию, особенно в приложениях с большим количеством независимых разделов.

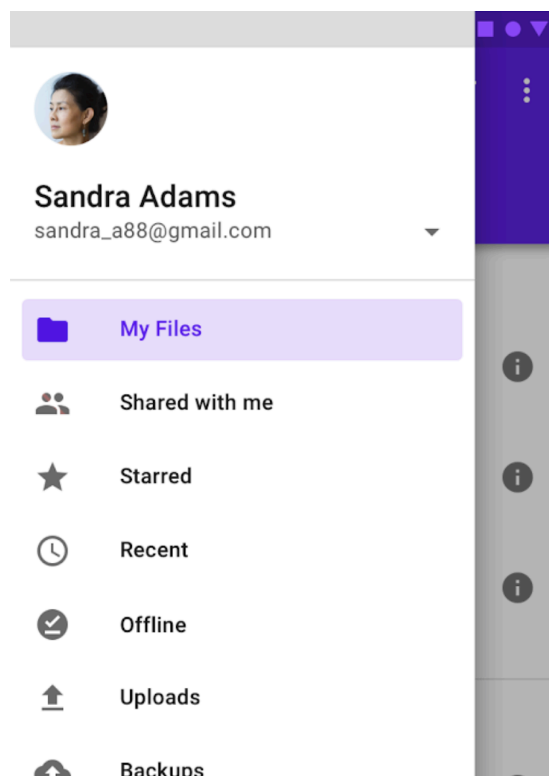


Рисунок 5. Пример интерфейса с Navigation Drawer

Рассмотрим основные элементы Navigation Drawer.

DrawerLayout - главный контейнер, который объединяет Navigation Drawer и основной контент приложения. Он управляет выдвижной панелью и обеспечивает плавное открытие/закрытие меню, занимает все пространство экрана, разделяя область на основной контент и меню, которое выдвигается сбоку.

NavigationView - специализированный виджет для создания бокового меню. Располагается внутри DrawerLayout и содержит элементы навигации (пункты меню, иконки, заголовки).

Header - верхняя часть Navigation Drawer, которая может содержать информацию о пользователе, логотип приложения.

Элементы меню включают текст, иконки и разделители и обрабатываются через слушатели событий.

Основной контент отображает содержимое приложения. Navigation Drawer взаимодействует с этой областью, заменяя или обновляя её содержимое в зависимости от выбранного пункта меню.

Кнопка «гамбургер» - элемент в верхней панели приложения, который открывает или закрывает Navigation Drawer. Его иконка часто отображается в виде трех горизонтальных линий.

Настройка Navigation Drawer начинается с определения разметки, где задаются области для основного содержимого и бокового меню. В меню можно добавлять пункты, каждому из которых назначается уникальный идентификатор и иконка. Далее в коде приложения реализуется логика взаимодействия, позволяющая открывать соответствующие разделы приложения при выборе пунктов меню. Navigation Drawer поддерживает ряд функций для улучшения пользовательского опыта: шторка может открываться и закрываться как программно, так и жестами пользователя, корректная реализация кнопки «Back» обеспечит закрытие шторки, а не выход из приложения.

Как и ранее подготовим 3 фрагмента. Для перехода между ними будет реализована навигация. Аналогично подготовим файлы *menu* и *navigation*. В файле *content_main.xml* опишем основное содержимое экрана, которое будет меняться в зависимости от выбранного пункта меню.

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
app:layout_behavior="@string/appbar_scrolling_view_behavior"
tools:showIn="@layout/app_bar_main">
<fragment
    android:id="@+id/nav_host"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
```

```
        app:navGraph="@navigation/navigation" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

В файле *app_bar_main.xml* опишем Toolbar и элемент для отображения содержимого. CoordinatorLayout является контейнером для элементов, *include* подключает разметку *content_main.xml*.

```
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/Theme.Navigation.AppBarOverlay">
        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="?attr/colorPrimary"
            app:popupTheme="@style/Theme.Navigation.PopupOverlay" />
        </com.google.android.material.appbar.AppBarLayout>

        <include layout="@layout/content_main" />
    </androidx.coordinatorlayout.widget.CoordinatorLayout>
```

В файле *nav_header_main.xml* опишем макет шапки Navigation Drawer, которая будет отображаться в верхней части шторки.

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="176dp"
    android:background="#009688"
    android:gravity="bottom"
    android:orientation="vertical"
    android:padding="16dp"
    android:theme="@style/ThemeOverlay.AppCompat.Dark">
    <ImageView
        android:layout_width="wrap_content"
```

```

        android:layout_height="wrap_content"
        android:paddingTop="8dp"
        app:srcCompat="@mipmap/ic_launcher_round" />
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingTop="8dp"
    android:text="Title" />
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Subtitle" />
</LinearLayout>

```

Теперь опишем разметку Activity. Она состоит из DrawerLayout, который включает *app_bar_main* и NavigationView.

```

<androidx.drawerlayout.widget.DrawerLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/drawer_layout"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:fitsSystemWindows="true"
tools:openDrawer="start">

    <include
        android:id="@+id/app_bar_main"
        layout="@layout/app_bar_main"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        android:fitsSystemWindows="true"
        app:headerLayout="@layout/nav_header_main"
        app:menu="@menu/menu" />
</androidx.drawerlayout.widget.DrawerLayout>

```

В MainActivity реализуем навигацию.

```

public class MainActivity extends AppCompatActivity {
    private AppBarConfiguration appBarConfiguration;
    private ActivityMainBinding binding;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());
        setSupportActionBar(binding.appBarMain.toolbar);

        appBarConfiguration = new AppBarConfiguration.Builder(
            R.id.nav_home, R.id.nav_info, R.id.nav_profile)
            .setOpenableLayout(binding.drawerLayout)
            .build();
        NavController navController = Navigation
            .findNavController(this, R.id.nav_host);
        NavigationUI.setupActionBarWithNavController(
            this, navController, appBarConfiguration);
        NavigationUI.setupWithNavController(
            binding.navView, navController);
    }

    @Override
    public boolean onSupportNavigateUp() {
        NavController navController = Navigation
            .findNavController(this, R.id.nav_host);
        return NavigationUI.navigateUp(navController,
            appBarConfiguration)
            || super.onSupportNavigateUp();
    }
}

```

Для создания проекта с использованием Navigation Drawer средствами IDE можно воспользоваться «New - New module - Navigation Drawer Views Activity».

Задание: Продумайте концепцию приложения, реализуйте его навигацию с помощью Navigation Drawer, обновите цветовую гамму, реализуйте закрытие шторки на кнопку «Back». Название модуля «NavigationDrawerApp».

7. КОНТРОЛЬНОЕ ЗАДАНИЕ

Задание: Требуется открыть приложение из практической работы №1. Реализовать в приложении навигацию с использованием Navigation Component и Navigation Drawer/Bottom Navigation.